In [2]:
```
import tensorflow as tf
import matplotlib.pyplot as plt

from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
```

In [3]:
```
(x_train, y_train), (x_test, y_test) = mnist.load_data()

print("Training images:", x_train.shape)
print("Testing images :", x_test.shape)
```

```
Training images: (60000, 28, 28)
Testing images : (10000, 28, 28)
```

In [6]:
```
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

In [5]:
```
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

print("Training shape:", x_train.shape)
print("Testing shape :", x_test.shape)
```

```
Training shape: (60000, 28, 28, 1)
Testing shape : (10000, 28, 28, 1)
```

In [7]:
```
model = models.Sequential([

    layers.Conv2D(
        32,
        (3, 3),
        activation="relu",
        input_shape=(28, 28, 1)
    ),

    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(
        64,
        (3, 3),
        activation="relu"
    ),

    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(
        128,
        (3, 3),
        activation="relu"
    ),

    layers.Flatten(),

    layers.Dense(
        128,
        activation="relu"
    ),

    layers.Dropout(0.5),

    layers.Dense(
        10,
        activation="softmax"
    )
])
```

```
C:\Users\cs2\AppData\Roaming\Python\Python313\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [8]:
```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape
conv2d (Conv2D)	(None, 26, 26, 32)
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)
conv2d_1 (Conv2D)	(None, 11, 11, 64)
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)
conv2d_2 (Conv2D)	(None, 3, 3, 128)
flatten (Flatten)	(None, 1152)
dense (Dense)	(None, 128)
dropout (Dropout)	(None, 128)
dense_1 (Dense)	(None, 10)

Total params: 241,546 (943.54 KB)

Trainable params: 241,546 (943.54 KB)

Non-trainable params: 0 (0.00 B)

In [9]:
```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
```

In [10]:
```
history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=128,
```

```
    validation_split=0.1
)
```

```
Epoch 1/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 10s 21ms/step - accuracy: 0.5618 - loss: 1.2318 - val_accuracy: 0.9060 - val_loss: 0.3268
Epoch 2/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 8s 20ms/step - accuracy: 0.8661 - loss: 0.4316 - val_accuracy: 0.9352 - val_loss: 0.2013
Epoch 3/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 9s 20ms/step - accuracy: 0.9122 - loss: 0.2880 - val_accuracy: 0.9592 - val_loss: 0.1369
Epoch 4/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 9s 20ms/step - accuracy: 0.9369 - loss: 0.2161 - val_accuracy: 0.9697 - val_loss: 0.1024
Epoch 5/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 9s 22ms/step - accuracy: 0.9488 - loss: 0.1724 - val_accuracy: 0.9752 - val_loss: 0.0869
```

In [11]:
```
test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print("================================")
print("CNN RESULTS")
print("================================")

print("Test Loss     :", test_loss)
print("Test Accuracy :", test_accuracy)
```

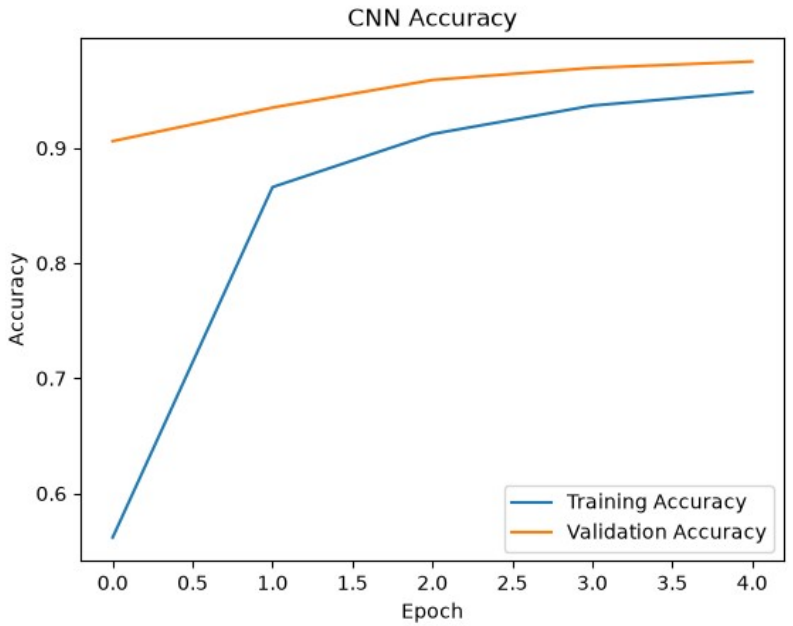
```
================================
CNN RESULTS
================================
Test Loss     : 0.08611845970153809
Test Accuracy : 0.972000002861023
```

In [12]:
```
plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title("CNN Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.show()
```

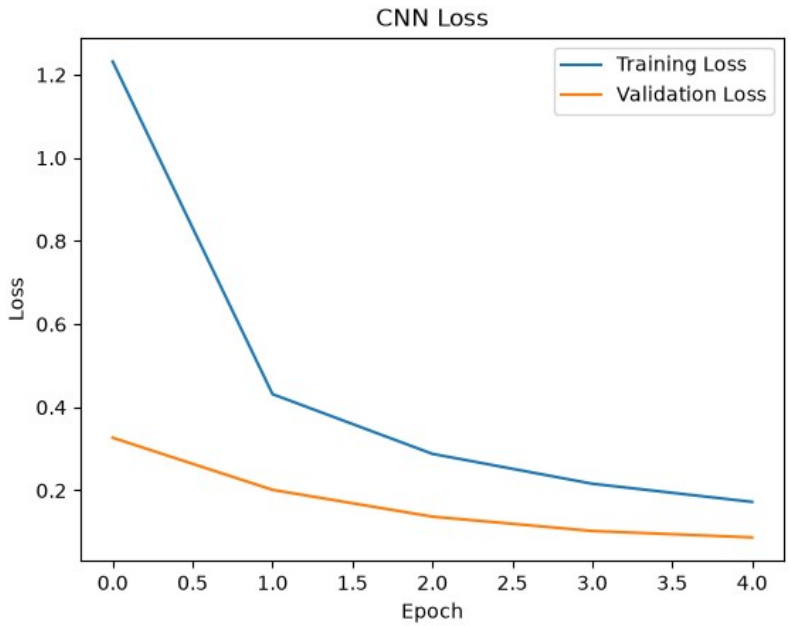


In [13]:
```
plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.title("CNN Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.show()
```



In [14]:
```python
import numpy as np

prediction = model.predict(x_test)

predicted_digit = np.argmax(prediction[0])

print("Predicted digit:", predicted_digit)
print("Actual digit   :", y_test[0])
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
Predicted digit: 7
Actual digit   : 7
```

In [16]:
```python
import numpy as np
import matplotlib.pyplot as plt

prediction = model.predict(x_test)

predicted_digits = np.argmax(prediction, axis=1)
actual_digits = y_test

plt.figure(figsize=(12, 8))

for i in range(12):
    plt.subplot(3, 4, i + 1)

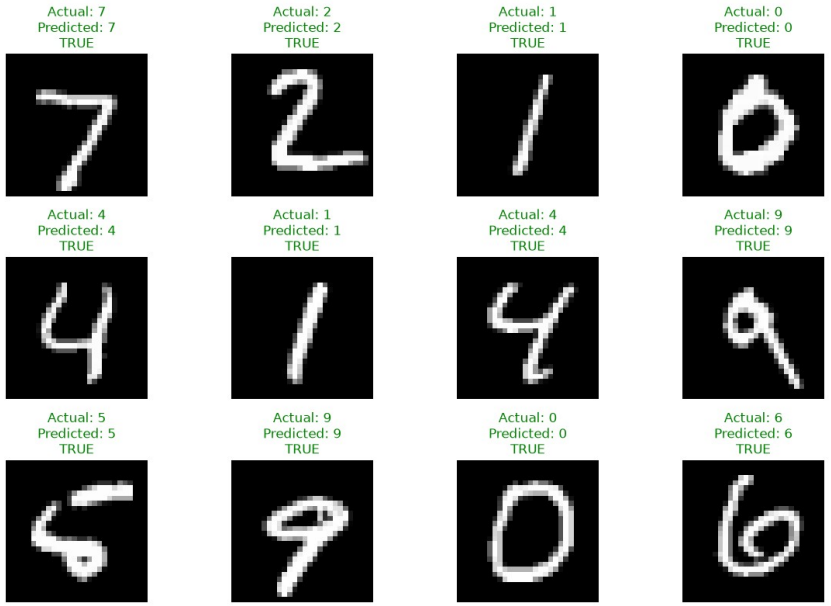
    actual = actual_digits[i]
    predicted = predicted_digits[i]

    if actual == predicted:
        result = "TRUE"
        color = "green"
    else:
        result = "FALSE"
        color = "red"

    plt.imshow(x_test[i][:, :, 0], cmap="gray")
    plt.title(
        f"Actual: {actual}\nPredicted: {predicted}\n{result}",
        color=color
    )
    plt.axis("off")

plt.tight_layout()
plt.show()
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step
```



In [ ]: